

Reviewer Pack — Minimal Gromov-Wasserstein Distance and DPLL Proof Path Leng...

Entry a37a8b3c04d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-07 16:14:20 UTC

Minimal Gromov-Wasserstein Distance and DPLL Proof Path Length

Entry ID: a37a8b3c04d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-07 16:14:20 UTC

1. Conjecture

Field A (mathematical branch): Metric Geometry (Gromov-Wasserstein Distance) **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For every Boolean satisfiability problem with n variables, the minimal Gromov-Wasserstein distance between the instance and a uniform distribution over all possible assignments is upper-bounded by the DPLL proof path length, i.e., $E[\text{GW_dist}(\text{instance})] \leq \Theta(\text{DPLL_path_length}(n))$.

Rationale (proposer's reasoning):

The Gromov-Wasserstein distance provides a way to measure the similarity of two probability measures in a metric space. If there is a strong correlation between this distance and the complexity of finding a proof, it could indicate that certain types of instances are more difficult to solve than others due to their structural properties.

Taxonomy category: metric_geometry_vs_dpll_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b4eb77f0e04157f9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Pearson correlation coefficient of the $\text{GW_dist}(\text{instance})$ and $\text{DPLL_path_length}(n)$ is ≥ 0.8 for at least 80% of the seeds, with no seed having a $\text{GW_dist}(\text{instance}) > 10$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Gromov-Wasserstein distance" AND "DPLL proof complexity" - "metric geometry" AND "Boolean satisfiability" AND Gromov-Wasserstein - "uniform distribution" IN BOOLEAN Satisfiability PROOF AND Gromov-Wasserstein

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/0903.3848v1] Join-irreducible Boolean functions - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def generate_instance(n):
    variables = list(range(n))
    clauses = []
    for _ in range(2 * n): # Each variable appears in at least two clauses
        clause = random.sample(variables, 3)
        if random.choice([True, False]):
            clause = [-v for v in clause]
        clauses.append(clause)
    return variables, clauses

def construct_metric_space(instance):
    variables, clauses = instance
    n = len(variables)
    points = [[Fraction(1, 2)] * n] # Uniform distribution over all assignments
    weights = []
    for clause in clauses:
        weight = sum(1 if v in clause else -1 for v in variables) / n
        weights.append(weight)
    return points, weights

def gaussian_elimination(A, b):
    n = len(b)
    augmented_matrix = [A[i] + [b[i]] for i in range(n)]

    # Forward elimination
```

```

for i in range(n):
    if augmented_matrix[i][i] == 0:
        for j in range(i+1, n):
            if augmented_matrix[j][i] != 0:
                augmented_matrix[i], augmented_matrix[j] = augmented_matrix[j], augmented_matrix[i]
                break
        else:
            raise ValueError("Matrix is singular")
    pivot = augmented_matrix[i][i]
    for j in range(n + 1):
        augmented_matrix[i][j] /= pivot

# Backward substitution
x = [0] * n
for i in range(n-1, -1, -1):
    sum_val = sum(augmented_matrix[i][j] * x[j] for j in range(i+1, n))
    x[i] = (augmented_matrix[i][n] - sum_val) / augmented_matrix[i][i]
return x

def gw_distance(instance):
    points, weights = construct_metric_space(instance)
    n = len(points[0])
    A = [[0] * n for _ in range(n)]
    b = [0] * n

    for i in range(n):
        for j in range(i+1, n):
            dist = sum((points[i][k] - points[j][k])**2 for k in range(n))
            A[i][j] = A[j][i] = dist
            b[i] += weights[j]
            b[j] += weights[i]

    x = gaussian_elimination(A, b)
    gw_dist = sum(x[i]**2 * weights[i] for i in range(n)) / n
    return gw_dist

def dpll_path_length(instance):
    variables, clauses = instance

    def is_satisfiable(assignment):
        for clause in clauses:
            if not any(v in assignment and (v > 0) == (c in assignment[v]) or v < 0 == (-c in assignment[v]) for v in clause):
                return False
        return True

    def backtrack(assignment, unassigned):
        if not unassigned:
            return is_satisfiable(assignment)

        var = unassigned[0]
        rest = unassigned[1:]
        assignment[var] = 1
        if backtrack(assignment, rest):
            return True
        del assignment[var]
        assignment[-var] = -1

```

```

        return backtrack(assignment, rest)

    return len(backtrack({}, variables))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    gw_dists = []
    dpll_lengths = []

    for n in n_values:
        instance = generate_instance(n)
        gw_dist = gw_distance(instance)
        if gw_dist > 10:
            return {
                "metric_name": "GW_dist",
                "metric_value": gw_dist,
                "instances_tested": 1,
                "n_max": n,
                "conjecture_holds": False,
                "counterexample": f"GW_dist(instance) = {gw_dist} > 10"
            }
        gw_dists.append(gw_dist)

        dpll_length = dpll_path_length(instance)
        dpll_lengths.append(dpll_length)

    correlation_coefficient = sum((gw_dists[i] - sum(gw_dists) / len(gw_dists)) * (dpll_lengths[i] - sum(dpll_lengths) / len(dpll_lengths))) / len(gw_dists)

    return {
        "metric_name": "GW_dist",
        "metric_value": correlation_coefficient,
        "instances_tested": len(gw_dists),
        "n_max": max(n_values),
        "conjecture_holds": correlation_coefficient >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])

```

```

    print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_s
else:
    print("RESULT: INCONCLUSIVE insufficient_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd5ac87.py", line 143, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd5ac87.py", line 111, in run_trial
    gw_dist = gw_distance(instance)
              ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd5ac87.py", line 70, in gw_distance
    dist = sum((points[i][k] - points[j][k])**2 for k in range(n))
              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_bfd5ac87.py", line 70, in <genexpr>
    dist = sum((points[i][k] - points[j][k])**2 for k in range(n))
                  ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing any data, which means that the pre-registered support condition could not be unambiguously met. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12901
2	preregistration	ollama_remote	glm4:latest	0	0	9534
3	novelty	ollama_remote	glm4:latest	0	0	8607
4	novelty	ollama_remote	glm4:latest	0	0	13873
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13286
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9682
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23578
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16986

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
9	judge	ollama_remote	glm4:latest	0	0	11665

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120111 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/a37a8b3c04d3.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a37a8b3c04d3.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a37a8b3c04d3.tar.gz (if generated)